

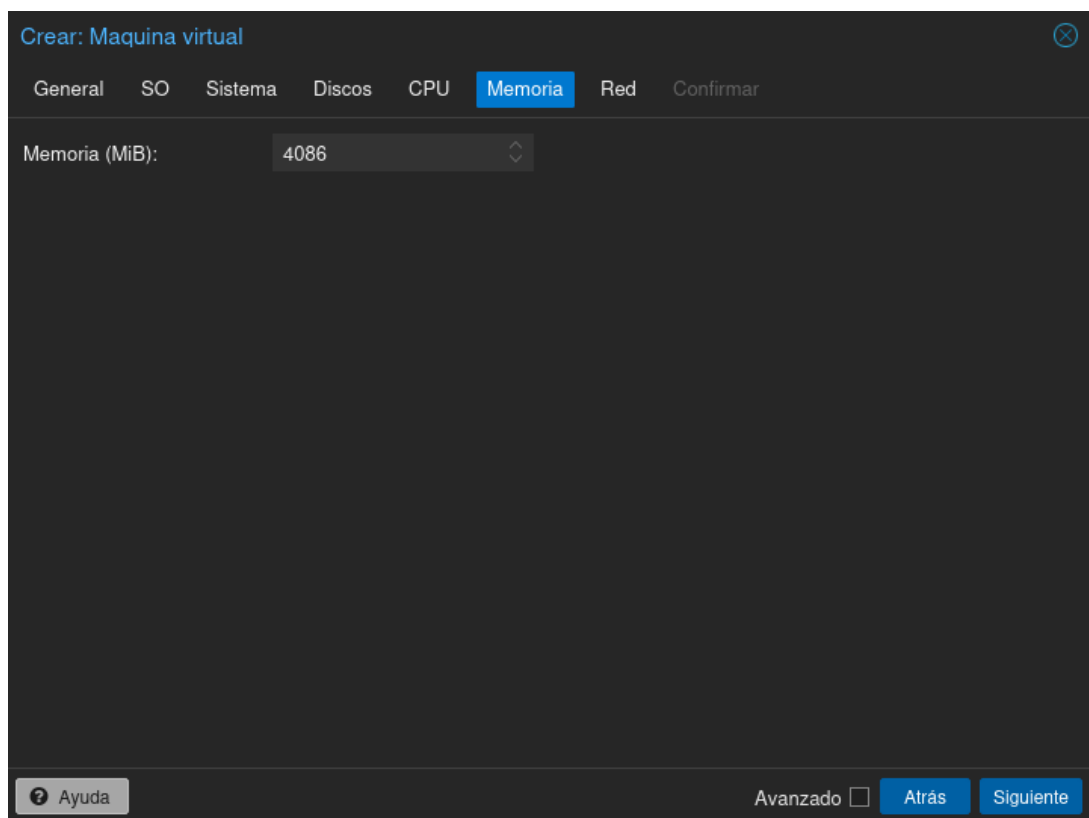
Documentación Técnica - Honeypot Lab

Instalación Visual - Honeypot Lab

Documentación del proceso de instalación con capturas visuales.

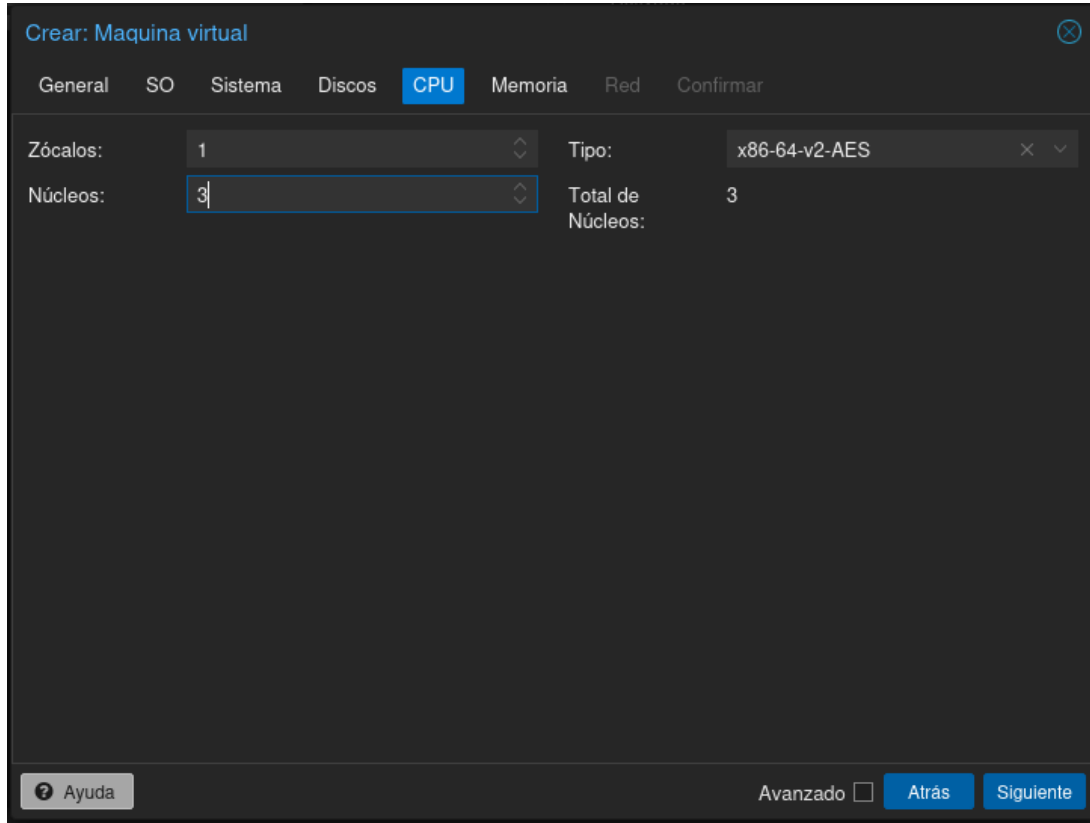
Paso 1: Asignación de Memoria

Se configura la máquina virtual con **4 GB de RAM** para que tenga suficientes recursos para ejecutar PostgreSQL, los honeypots y el sistema operativo simultáneamente.



Paso 2: Configuración de CPU

Se asigna **3 núcleos de CPU** en el tipo x86-64-v2-AES, proporcionando poder de procesamiento adecuado para manejar múltiples conexiones simultáneas de atacantes.



The screenshot shows the 'Crear: Maquina virtual' (Create: Virtual Machine) window in Proxmox VE, specifically the 'CPU' configuration tab. The window has a dark theme and a top navigation bar with tabs: General, SO, Sistema, Discos, CPU (selected), Memoria, Red, and Confirmar. The CPU configuration section shows 'Zócalos' (Sockets) set to 1 and 'Núcleos' (Cores) set to 3. The 'Tipo' (Type) is set to 'x86-64-v2-AES'. Below these, it shows 'Total de Núcleos' (Total of Cores) as 3. At the bottom, there is an 'Ayuda' (Help) button, an 'Avanzado' (Advanced) checkbox which is unchecked, and 'Atrás' (Back) and 'Siguiente' (Next) buttons.

Configuration Item	Value
Zócalos	1
Núcleos	3
Tipo	x86-64-v2-AES
Total de Núcleos	3

Paso 3: Configuración de Disco

Se crea un disco virtual de **40 GB** con almacenamiento local en Proxmox. Este espacio es suficiente para el SO, PostgreSQL y los logs del honeypot.

Crear: Maquina virtual

General

SO

Sistema

Discos

CPU

Memoria

Red

Confirmar

scsi0

Disco

Ancho de banda

Bus/Dispositivo:

SCSI

0

Caché:

Por defecto (No hay)

Controlador SCSI:

VirtIO SCSI single

Descartar:

☐

IO thread:

☒

Almacenamiento:

local-lvm

Tamaño de disco (GiB):

40

Formato:

Imagen de disco en l

+

Agregar

📁

Importar

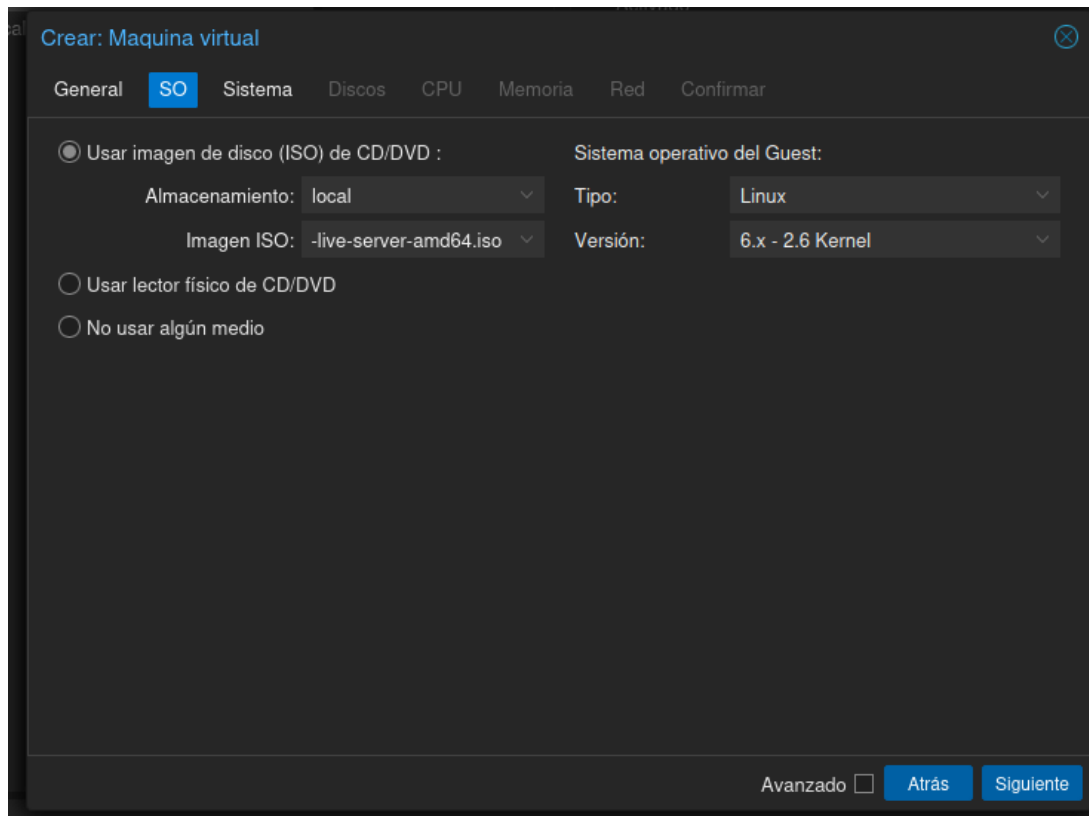
Avanzado ☐

Atrás

Siguiente

Paso 4: Selección del Sistema Operativo

Se elige **Ubuntu Server 22.04 LTS** como sistema operativo base. Esta distribución es estable, tiene soporte a largo plazo y amplio soporte comunitario para herramientas de seguridad.



Paso 5: Información General de la VM

Se define el nombre de la máquina como “**honeypot**” con ID 101 en el nodo Proxmox. Esto permite identificarla fácilmente en la infraestructura.

Crear: Maquina virtual

General

SO

Sistema

Discos

CPU

Memoria

Red

Confirmar

Nodo:

pve2

VM ID:

101

Nombre:

honeypot

Conjunto de recursos:

Ayuda

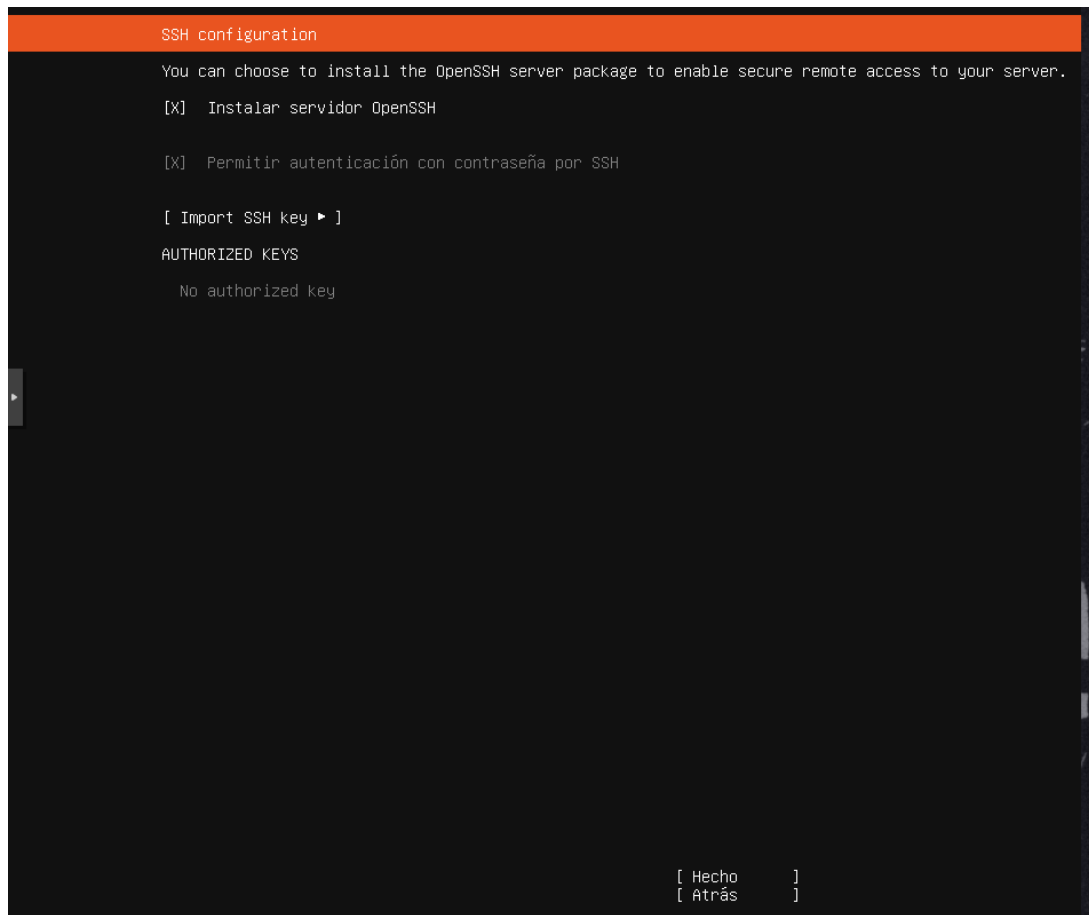
Avanzado

Atrás

Siguiente

Paso 6: Instalación de OpenSSH

Durante la instalación de Ubuntu, se selecciona instalar **OpenSSH Server** para permitir acceso remoto por SSH a la máquina. Esto es útil para gestionar el honeypot desde otras máquinas.



Paso 7: Configuración del Usuario

Se crea el usuario “**administrador**” con contraseña. Este usuario tendrá permisos sudo para instalar dependencias y configurar los servicios del honeypot.

Profile configuration

Enter the username and password you will use to log in to the system. You can configure SSH access or password is still needed for sudo.

Su nombre:

administrador

Your servers name:

honeypot

The name it uses when it talks to other computers.

Elija un nombre de usuario:

administrador

Elija una contraseña:

Confirme la contraseña:

[Hecho]

Paso 8: Instalación de PostgreSQL

Se instala **PostgreSQL** ejecutando el comando apt. PostgreSQL es necesario para almacenar todos los ataques capturados por los honeypots en una base de datos relacional.

```
administrador@honeypot:~/honeypot-lab$ sudo apt install postgresql postgresql-contrib
[sudo] password for administrador:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libcommon-sense-perl libjson-perl libjson-xs-perl liblvm17t64 libpq5 libtypes-serialiser-perl postgresql-16 postgresql-client-common postgresql-common postgresql-contrib ssl-cert
Paquetes sugeridos:
  postgresql-doc postgresql-doc-16
Se instalarán los siguientes paquetes NUEVOS:
  libcommon-sense-perl libjson-perl libjson-xs-perl liblvm17t64 libpq5 libtypes-serialiser-perl postgresql postgresql-client-common postgresql-common postgresql-contrib ssl-cert
0 actualizados, 13 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 43,6 MB de archivos.
Se utilizarán 175 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]
```

Paso 9: Verificar PostgreSQL Activo

Se verifica que el servicio PostgreSQL esté **activo y ejecutándose**. Un estado “Active: active (running)” confirma que la base de datos está lista para recibir conexiones.

```
administrador@honeypot:~/honeypot-lab$ sudo systemctl start postgresql
administrador@honeypot:~/honeypot-lab$ sudo systemctl enable postgresql
Synchronizing state of postgresql.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable postgresql
administrador@honeypot:~/honeypot-lab$ sudo systemctl status postgresql
● postgresql.service - PostgreSQL RDBMS
   Loaded: loaded (/usr/lib/systemd/system/postgresql.service; enabled; preset: enabled)
   Active: active (exited) since Sun 2026-05-24 20:18:26 UTC; 48s ago
     Main PID: 21536 (code=exited, status=0/SUCCESS)
        CPU: 1ms

May 24 20:18:26 honeypot systemd[1]: Starting postgresql.service - PostgreSQL RDBMS...
May 24 20:18:26 honeypot systemd[1]: Finished postgresql.service - PostgreSQL RDBMS.
administrador@honeypot:~/honeypot-lab$
```

Paso 10: Instalar Herramientas Adicionales

Se instala **postgresql-contrib** que proporciona herramientas y extensiones adicionales para PostgreSQL que pueden ser útiles para administración.

Paso 11: Crear Base de Datos

Se crea la base de datos “**honeypot_db**” en PostgreSQL. Esta BD almacenará todos los registros de ataques capturados por los honeypots.

```
administrador@honeypot:~/honeypot-lab$ sudo -u postgres psql
psql (16.14 (Ubuntu 16.14-0ubuntu0.24.04.1))
Type "help" for help.

postgres=# CREATE DATABASE honeypot_db;
CREATE DATABASE
```

Paso 12: Crear Usuario de Base de Datos

Se crea el usuario “**honeypot_user**” en PostgreSQL con contraseña. Este usuario será utilizado por los honeypots para conectarse a la base de datos de forma segura.

```
postgres=# CREATE USER honeypot_user WITH PASSWORD 'ejemplo';
CREATE ROLE
postgres=# ALTER ROLE honeypot_user SET client_encoding TO 'utf8';
ALTER ROLE
postgres=# GRANT ALL PRIVILEGES ON DATABASE honeypot_db TO honeypot_user;
ERROR:  syntax error at or near "PRIVILEGES"
LINE 1: GRANT ALL PRIVILEGES ON DATABASE honeypot_db TO honeypot_us...
                  ^
postgres=# GRANT ALL PRIVILEGES ON DATABASE honeypot_db TO honeypot_user;
GRANT
postgres=# _
```

Paso 13: Crear Estructura del Proyecto

Se crean los directorios y archivos iniciales del proyecto. Se organizan los módulos en carpetas separadas (honeypot, dashboard, tests) y se copian los archivos de configuración necesarios.

```
administrador@honeypot:~/honeypot-env$ nano config.py
administrador@honeypot:~/honeypot-env$ nano logger.py
administrador@honeypot:~/honeypot-env$ nano .env
administrador@honeypot:~/honeypot-env$ nano requirements.txt
administrador@honeypot:~/honeypot-env$ nano .gitignore
administrador@honeypot:~/honeypot-env$ touch honeypot/__init__.py
```

Paso 14: Instalar Herramientas del Sistema

Se instalan herramientas de utilidad del sistema que pueden ser necesarias para que ciertos paquetes Python compilen correctamente.

```

(honeypot-env) administrador@honeypot:~$ python -c "import flask, paramiko, pycopg2, sqlalchemy, loguru, requests;
todo funciona perfectamente
(honeypot-env) administrador@honeypot:~$ pip list
Package            Version
-----
bcrypt             5.0.0
clicnlp            1.9.0
certifi            2026.5.20
cffi               2.0.0
charset-normalizer 3.4.7
click              8.4.1
cryptography       48.0.0
flask              3.1.3
greenlet           3.5.1
idna               3.16
invoke             3.0.3
itsdangerous       2.2.0
jinja2             3.1.6
loguru             0.7.3
MarkupSafe         3.0.3
paramiko           5.0.0
pip               26.1.1
pycpg2-binary      2.9.12
pycparser          3.0
pyNaCl             1.6.2
python-dotenv      1.2.2
requests           2.34.2
SQLAlchemy         2.0.49
typing_extensions 4.15.0
urllib3            2.7.0
werkzeug           3.1.8
(honeypot-env) administrador@honeypot:~$ deactivate
administrador@honeypot:~$ ^C
administrador@honeypot:~$

```

Paso 15: Verificar Dependencias Python

Se lista todos los paquetes Python instalados. Se pueden ver librerías críticas como Flask, Paramiko, pycopg2, Loguru y Requests que son necesarias para el proyecto.

```

administrador@honeypot:~$ sudo apt install python3.12-venv python3.12-dev
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  libexpat1-dev libpython3.12-dev python3-pip-whl python3-setuptools-whl zlib1g-dev
Se instalarán los siguientes paquetes NUEVOS:
  libexpat1-dev libpython3.12-dev python3-pip-whl python3-setuptools-whl python3.12-dev python3.12-venv zlib1g-dev
0 actualizados, 7 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 9.643 kB de archivos.
Se utilizarán 35,2 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n]

```

Paso 16: Actualizar pip

Se actualiza **pip** (gestor de paquetes Python) a la versión más reciente. Esto asegura que se instalen correctamente todas las dependencias del proyecto.

```
administrador@honeypot:~$ python3 -m venv ~/honeypot-env
administrador@honeypot:~$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~$ pip install --upgrade pip
Requirement already satisfied: pip in ./honeypot-env/lib/python3.12/site-packages (24.0)
Collecting pip
  Downloading pip-26.1.1-py3-none-any.whl.metadata (4.6 kB)
  Downloading pip-26.1.1-py3-none-any.whl (1.8 MB)
    1.8/1.8 MB 12.6 MB/s eta 0:00:00
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 24.0
    Uninstalling pip-24.0:
      Successfully uninstalled pip-24.0
Successfully installed pip-26.1.1
(honeypot-env) administrador@honeypot:~$ pip install \
> flask \
> paramiko \
> pycopg2-binary \
> sqlalchemy \
> python-dotenv \
> loguru \
> requests_
```

Paso 17: Instalar Python 3.12 Dev

Se instalan los **headers de desarrollo de Python 3.12**. Esto es necesario para compilar módulos nativos de algunos paquetes como pycopg2.

```
(honeypot-env) administrador@honeypot:~$ pip install \
> geoip2 \
> maxminddb \
> scrapy \
> dnspython \
> pycryptodome \
> pytest \
> pytest-cov_
```

Paso 18: Instalar Utilidades Básicas

Se instalan herramientas del sistema como **git**, **curl**, **nano**, **vim**, **htop** y otras. Estas son fundamentales para desarrollo, debugging y monitoreo del servidor.

```
administrador@honeypot:~$ sudo apt install -y \  
> git\  
> curl\  
> wget\  
> nano\  
> vim\  
> htop\  
> net-tools\  
> build-essential\  
> software-properties-common _
```

Paso 19: SSH Honeypot Funcionando

El SSH Honeypot se inicia correctamente en puerto 2222 y comienza a aceptar conexiones. Se ve que la BD está lista y se registran múltiples conexiones de atacantes. Cada intento de login se captura y se clasifica por tipo de ataque (LOGIN_ATTEMPT, BRUTE_FORCE).

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate  
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 ssh_trap.py  
INFO | __main__:main:295 - =====  
INFO | __main__:main:296 - HONEYPOT-LAB - SSH HONEYPOT v0.1 (CON BD)  
INFO | __main__:main:297 - =====  
INFO | __main__:main:298 - Inicializando SSH Honeypot...  
INFO | __main__:main:299 - Host: 0.0.0.0  
INFO | __main__:main:300 - Puerto: 2222  
INFO | __main__:main:301 - Banner: SSH-2.0-OpenSSH_7.4  
INFO | __main__:main:302 - Max intentos auth: 3  
INFO | __main__:main:303 - =====  
INFO | __main__:main:306 -  
♦ Inicializando base de datos...  
INFO | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db  
INFO | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente  
INFO | logger:log_db_event:50 - Tabla inicializada | attacks  
INFO | __main__:main:309 - ♦ Base de datos lista  
  
INFO | __main__:start:208 - ♦ SSH Honeypot iniciado en 0.0.0.0:2222 (Banner: SSH-2.0-OpenSSH_7.4)  
INFO | __main__:accept_connections:234 - ♦ Cliente #1 conectado: 100.111.145.79  
INFO | logger:log_connection:46 - Conexión | IP: 100.111.145.79 | Servicio: SSH | Puerto: 2222  
INFO | __main__:__init__:45 - ♦ Conexión SSH recibida desde 100.111.145.79:60888  
WARNING | __main__:check_auth_password:56 - ♦ SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass:  
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 1 | IP: 100.111.145.79 | Tipo: LOGIN_ATTEMPT  
INFO | __main__:check_auth_password:90 - ♦ Ataque guardado en BD | ID: 1  
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: LOGIN_ATTEMPT  
WARNING | __main__:check_auth_password:56 - ♦ SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass:  
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 2 | IP: 100.111.145.79 | Tipo: BRUTE_FORCE  
INFO | __main__:check_auth_password:90 - ♦ Ataque guardado en BD | ID: 2  
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: BRUTE_FORCE  
WARNING | __main__:check_auth_password:56 - ♦ SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass:  
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 3 | IP: 100.111.145.79 | Tipo: BRUTE_FORCE  
INFO | __main__:check_auth_password:90 - ♦ Ataque guardado en BD | ID: 3  
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: BRUTE_FORCE  
CRITICAL | __main__:check_auth_password:106 - ♦ BRUTE FORCE CRÍTICO DETECTADO | IP: 100.111.145.79 | Intentos: 3
```

Paso 20: HTTP Honeypot Capturando Ataques

El HTTP Honeypot se inicia en puerto 8080 y comienza a detectar ataques. Se capturan diversos tipos de ataques: PATH_TRAVERSAL, SQL_INJECTION, XSS. Cada ataque se inserta en la BD con un ID único y tipo de amenaza.

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 ssh_trap.py
INFO | __main__:main:295 - =====
INFO | __main__:main:296 - HONEYPOT-LAB - SSH HONEYPOT v0.1 (CON BD)
INFO | __main__:main:297 - =====
INFO | __main__:main:298 - Inicializando SSH Honeypot...
INFO | __main__:main:299 - Host: 0.0.0.0
INFO | __main__:main:300 - Puerto: 2222
INFO | __main__:main:301 - Banner: SSH-2.0-OpenSSH_7.4
INFO | __main__:main:302 - Max intentos auth: 3
INFO | __main__:main:303 - =====
INFO | __main__:main:306 -
* Inicializando base de datos...
INFO | database:connect:39 - * Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - * Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:309 - * Base de datos lista

INFO | __main__:start:208 - * SSH Honeypot iniciado en 0.0.0.0:2222 (Banner: SSH-2.0-OpenSSH_7.4)
INFO | __main__:accept_connections:234 - * Cliente #1 conectado: 100.111.145.79
INFO | logger:log_connection:46 - Conexión | IP: 100.111.145.79 | Servicio: SSH | Puerto: 2222
INFO | __main__:__init__:45 - * Conexión SSH recibida desde 100.111.145.79:60888
WARNING | __main__:check_auth_password:56 - * SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass:
INFO | database:insert_attack:147 - * Ataque insertado | ID: 1 | IP: 100.111.145.79 | Tipo: LOGIN_ATTEMPT
INFO | __main__:check_auth_password:90 - * Ataque guardado en BD | ID: 1
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: LOGIN_ATTEMPT
WARNING | __main__:check_auth_password:56 - * SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass:
INFO | database:insert_attack:147 - * Ataque insertado | ID: 2 | IP: 100.111.145.79 | Tipo: BRUTE_FORCE
INFO | __main__:check_auth_password:90 - * Ataque guardado en BD | ID: 2
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: BRUTE_FORCE
WARNING | __main__:check_auth_password:56 - * SSH LOGIN ATTEMPT | IP: 100.111.145.79 | Username: administrador | Pass:
INFO | database:insert_attack:147 - * Ataque insertado | ID: 3 | IP: 100.111.145.79 | Tipo: BRUTE_FORCE
INFO | __main__:check_auth_password:90 - * Ataque guardado en BD | ID: 3
WARNING | logger:log_attack:39 - ATAQUE DETECTADO | IP: 100.111.145.79 | Servicio: SSH | Tipo: BRUTE_FORCE
CRITICAL | __main__:check_auth_password:106 - * BRUTE FORCE CRITICO DETECTADO | IP: 100.111.145.79 | Intentos: 3
```

Paso 21: Menú del Simulador de Ataques

El simulador de ataques presenta un menú interactivo con 6 opciones diferentes. Permite generar ataques SSH, HTTP, mixtos o ejecutar tests de stress sin exponerse a ataques reales.

```

administrador@honeypot:~/honeypot-lab/tests$ cd
administrador@honeypot:~$ cd honeypot-lab/
administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 tests/simulate_attacks.py
=====
HONEYPOT-LAB - SIMULADOR DE ATAQUES
=====

Opciones:
1. Simular ataque SSH (fuerza bruta)
2. Simular ataques HTTP (SQL, XSS, etc)
3. Simular ataques mixtos (SSH + HTTP)
4. Ataque rapido (5 SSH + 10 HTTP)
5. Estres test (100 requests)
0. Salir

Selecciona opcion: _

```

Paso 22: Ejecutar Test Rápido

Se ejecuta la **opción 4 (quick test)** que genera 5 ataques SSH y 10 ataques HTTP automáticamente. Esto llena la base de datos con datos de prueba para validar que los honeypots capturan correctamente.

```

Opciones:
1. Simular ataque SSH (fuerza bruta)
2. Simular ataques HTTP (SQL, XSS, etc)
3. Simular ataques mixtos (SSH + HTTP)
4. Ataque rapido (5 SSH + 10 HTTP)
5. Estres test (100 requests)
0. Salir

Selecciona opcion: 4

[QUICK] Ejecutando ataque rapido...

[SSH] Iniciando simulacion de 5 intentos de login...
Error en intento 1: [Errno 111] Connection refused
Error en intento 2: [Errno 111] Connection refused
Error en intento 3: [Errno 111] Connection refused
Error en intento 4: [Errno 111] Connection refused
Error en intento 5: [Errno 111] Connection refused
[SSH] Simulacion completada: 5 intentos

[HTTP] Iniciando simulacion de 10 requests...
Error en request 1: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /search?q=%3Csvg%
onError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection r
Error en request 2: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /api/users?id=15
used by NewConnectionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno
Error en request 3: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /view?page=../..
or("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused")
Error en request 4: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /read?path=../..
HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused"))
Error en request 5: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /get?file=..%2F..
ionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection r
Error en request 6: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /input?data=%3C%
ectionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection
Error en request 7: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /get?file=..%2F..
ionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection r
Error en request 8: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /comment?text=%3
by NewConnectionError("HTTPConnection(host='localhost', port=8080): Failed to establish a new connection: [Errno 111]
Error en request 9: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /wp-admin (Cause
st='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused"))
Error en request 10: HTTPConnectionPool(host='localhost', port=8080): Max retries exceeded with url: /phpmyadmin (Ca
host='localhost', port=8080): Failed to establish a new connection: [Errno 111] Connection refused"))

[HTTP] Simulacion completada:
- SQL_INJECTION: 0 requests
- XSS: 0 requests
- PATH_TRAVERSAL: 0 requests
- SCANNER: 0 requests

[QUICK] Ataque rapido completado
(honeypot-env) administrador@honeypot:~/honeypot-lab$ _

```

Paso 23: Código del Simulador

Se muestra el código del **simulador de ataques** que contiene configuración, diccionarios de usuarios/contraseñas para SSH y payloads de ataques web (SQL injection, XSS, path traversal, scanners).

```
GNU nano 7.2 simulate_atacks.py
.....
Simulador de ataques para honeypot-lab
Genera ataques SSH y HTTP para testing
.....

import socket
import time
import random
import requests
from datetime import datetime

# Configuración
SSH_HOST = "localhost"
SSH_PORT = 2222
HTTP_HOST = "localhost"
HTTP_PORT = 8080

# Datos de prueba
SSH_USERS = [
    "admin", "root", "test", "user", "administrator",
    "postgres", "mysql", "oracle", "ubuntu", "pi"
]

SSH_PASSWORDS = [
    "admin", "password", "123456", "password123", "admin123",
    "root", "12345678", "querty", "test", "letmein"
]

HTTP_PAYLOADS = {
    "SQL_INJECTION": [
        "/index.php?id=1' OR '1'='1",
        "/login.php?user=admin'--",
        "/search?q='; DROP TABLE users--",
        "/api/users?id=1 UNION SELECT * FROM passwords",
        "/product.php?id=1' AND 1=1--"
    ],
    "XSS": [
        "/search?q=<script>alert(1)</script>",
        "/comment?text=<img src=x onerror=alert(1)>",
        "/page?name=<script>document.cookie</script>",
        "/input?data=<iframe src=evil.com>",
        "/search?q=<svg onload=alert(1)>"
    ],
    "PATH_TRAVERSAL": [
        "/download?file=../../../../etc/passwd",
        "/read?path=../../../../etc/shadow",
    ]
}
```

Paso 24: HTTP Honeypot en Puerto 8080

Se confirma que el **HTTP Honeypot** está ejecutándose en puerto 8080 con acceso a BD. Está listo para capturar ataques web.

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 http_trap.py
INFO | __main__:main:219 - =====
INFO | __main__:main:220 - HONEYPOT-LAB - HTTP HONEYPOT v0.1
INFO | __main__:main:221 - =====
INFO | __main__:main:222 - Inicializando HTTP Honeypot...
INFO | __main__:main:223 - Host: 0.0.0.0
INFO | __main__:main:224 - Puerto: 8080
INFO | __main__:main:225 - =====
INFO | __main__:main:228 -
Verificando base de datos...
INFO | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:231 - Base de datos lista

INFO | __main__:start:45 - HTTP Honeypot iniciado en 0.0.0.0:8080
```

Paso 25: Ejecutar Ataque XSS con curl

Desde terminal se ejecuta un **ataque XSS** usando curl contra el honeypot HTTP. El honeypot responde con 200 OK, engañando al atacante de que el servidor es real.

```
administrador@honeypot:~/honeypot-lab$ curl "http://localhost:8080/search?q=<script>alert(1)</script>"
<html><body><h1>Welcome</h1></body></html>administrador@honeypot:~/honeypot-lab$ _
```

Paso 26: HTTP Honeypot Detecta Ataque

El honeypot **detecta y registra el ataque XSS** en los logs y lo inserta en la BD con ID único. Se identifica como ataque de tipo SQL_INJECTION (puede haber falsa clasificación en este caso).

```
(honeypot-env) administrador@honeypot:~/honeypot-lab$ source ~/honeypot-env/bin/activate
(honeypot-env) administrador@honeypot:~/honeypot-lab$ python3 http_trap.py
INFO | __main__:main:219 - =====
INFO | __main__:main:220 - HONEYPOT-LAB - HTTP HONEYPOT v0.1
INFO | __main__:main:221 - =====
INFO | __main__:main:222 - Inicializando HTTP Honeypot...
INFO | __main__:main:223 - Host: 0.0.0.0
INFO | __main__:main:224 - Puerto: 8080
INFO | __main__:main:225 - =====
INFO | __main__:main:228 -
Verificando base de datos...
INFO | database:connect:39 - ♦ Conexión a BD exitosa: localhost:5432/honeypot_db
INFO | database:init_database:408 - ♦ Tabla 'attacks' inicializada correctamente
INFO | logger:log_db_event:50 - Tabla inicializada | attacks
INFO | __main__:main:231 - Base de datos lista

INFO | __main__:start:45 - HTTP Honeypot iniciado en 0.0.0.0:8080
INFO | __main__:handle_request:109 - HTTP Request | IP: 127.0.0.1 | GET /search?q=<script>alert(1)</script>
INFO | database:insert_attack:147 - ♦ Ataque insertado | ID: 4 | IP: 127.0.0.1 | Tipo: SQL_INJECTION
INFO | __main__:handle_request:127 - Ataque HTTP guardado | ID: 4 | Tipo: SQL_INJECTION
INFO | logger:log_attack:39 - ATAQUE DETECTADO | IP: 127.0.0.1 | Servicio: HTTP | Tipo: SQL_INJECTION
```

Paso 27: Código HTTP Honeypot

Se muestra el código del **HTTP Honeypot** que implementa un servidor HTTP que captura requests, detecta patrones maliciosos y almacena los ataques en PostgreSQL usando la clase HTTPHoneypot.

```
GNU nano 7.2 http_trap.py
"""
HTTP Honeypot - Captura ataques web
Detecta SQL injection, XSS, path traversal y otros ataques HTTP
Guarda todos los ataques en PostgreSQL
"""

import socket
import threading
import time
import os
import sys
from datetime import datetime
from urllib.parse import unquote

# Agregar parent directory al path
sys.path.insert(0, os.path.dirname(os.path.dirname(os.path.abspath(__file__))))

from logger import logger, log_attack, log_connection, log_error
import config
import database

class HTTPHoneypot:
    """
    Servidor HTTP honeypot que captura ataques web
    """

    def __init__(self, host=config.HONEYPOT_HOST, port=config.HONEYPOT_HTTP_PORT):
        self.host = host
        self.port = port
        self.running = False
        self.server_socket = None
        self.request_count = 0

    def start(self):
        """Iniciar servidor HTTP"""
        try:
            self.server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
            self.server_socket.setsockopt(socket.SOL_SOCKET, socket.SO_REUSEADDR, 1)

            self.server_socket.bind((self.host, self.port))
            self.server_socket.listen(100)
            self.running = True

            logger.info(f"HTTP Honeypot iniciado en {self.host}:{self.port}")
        except Exception as e:
            logger.error(f"Error al iniciar el servidor HTTP: {e}")
```

Instalación completada exitosamente